JOHANNES KEPLER
UNIVERSITÄT LINZ

UniRoom - JKU Library Room Booker Calendar

367.022, KV Mobile Web Development with Python and JavaScript, Ismail Khalil, 2025W

Presented by: Daniel Serdeshnyi k12148401

JOHANNES KEPLER
UNIVERSITÄT LINZ
Altenberger Straße 69
4040 Linz, Österreich
jku.at

Project overview

What is it?

- Mobile-first web app to **browse free slots + book a room in one flow**
- Combines the current **availability calendar + booking form** into one UI

Who it’s for?

JKU students booking group study rooms (mobile first)

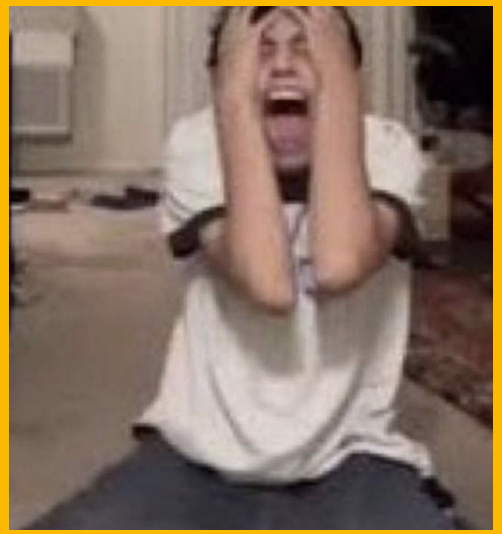
Why it matters?

Current system: hard to use on mobile + split across two sites → friction + mistakes

Problem → Goal → Key Features

Problems:

1. Calendar is painful on phones (tiny, awkward navigation)
2. Booking is on a separate site → extra steps + higher error rate



Problem → Goal → Key Features

Problems:

1. Calendar is painful on phones (tiny, awkward navigation)
2. Booking is on a separate site → extra steps + higher error rate

Goal

Reduce **time-to-book**

Prevent **double bookings** / conflicts

Make it **fully responsive** + PWA-ready

Problem → Goal → Key Features

Key features:

- Mobile calendar (day view, swipe/tap navigation)
- Free/busy timeline per room + conflict warning before submit
- One-tap booking drawer (review → confirm → success + email)
- Filters (date range, time window, capacity, equipment)
- My bookings + cancel, attendee emails

System Architecture

Frontend (HTML/CSS/JS + Bootstrap)

- Calendar UI + booking drawer + filters
- Fetches room + availability + bookings

Backend (Python/Django)

- REST-ish endpoints (or Django views returning JSON)
- Validation + conflict checks
- Email sending

Database (SQLite for demo)

Rooms, bookings, attendees, equipment

System Architecture: Flow

Frontend (HTML/CSS/JS + Bootstrap)

- Calendar UI + booking drawer + filters
- Fetches room + availability + bookings

Frontend requests availability

User selects slot

Backend (Python/Django)

- REST-ish endpoints (or Django views returning JSON)
- Validation + conflict checks
- Email sending

Backend validates (conflict + rules)

Database (SQLite for demo)

Rooms, bookings, attendees, equipment

DB write → return success

Emails sent → UI updates calendar instantly

Implementation Details

Frontend (HTML/CSS/JS)

- **Day view timeline**: time grid + overlays for “busy blocks”
- Touch-friendly navigation: swipe/tap to change day, quick jump to weeks
- Booking drawer: compact form + room details shown before confirmation
- Client-side guardrails:
  - highlight invalid ranges
  - clear error text from backend (conflict, invalid attendees, etc.)

Backend (Django)

- Endpoints like:
  - GET /api/rooms
  - GET /api/availability?date=...
  - POST /api/bookings
  - GET /api/my-bookings
  - DELETE /api/bookings/<id>
- Server-side rules:
  - no overlap booking per room
  - time window validation
  - attendee email list validation

Database: Room has capacity, equipment, location;

- Booking has room_id, start, end, owner, attendees

Codebase & Folder Structure

```
uniroom/
├── manage.py
├── db.sqlite3
├── .venv/                    # Python virtual environment (local only)
│
├── config/                   # Django project config
│   ├── __init__.py
│   ├── settings.py           # Django settings (static, email, allowed hosts, etc.)
│   ├── urls.py               # URL routing (includes /api + /sw.js)
│   ├── wsgi.py
│   └── asgi.py
```

Codebase & Folder Structure

```
├── booking/                    # Main app: rooms + bookings
│   ├── __init__.py
│   ├── admin.py                # Admin UI for Room/Booking/Attendee
│   ├── api.py                  # JSON API endpoints + session auth + emails
│   ├── models.py               # Room, Booking, Attendee models
│   ├── urls.py                 # App routes (API + index)
│   ├── views.py                # Serves index page
│   └── migrations/
│       ├── __init__.py
│       └── 0001_initial.py
│
├── templates/                  # Server-rendered HTML + service worker
│   ├── index.html              # UI (calendar, filters, login modal, drawer, my
bookings)
│   └── sw.js                   # Service worker served at /sw.js (PWA)
```

Codebase & Folder Structure

```
└── static/                          # Front-end assets
    ├── css/
    │   └── styles.css               # Mobile-first layout + calendar styling
    ├── js/
    │   └── app.js                   # Calendar UI, booking flow, filters, session auth, SW
register
    ├── icons/
    │   ├── icon-192.png             # PWA icon
    │   ├── icon-512.png             # PWA icon
    │   └── apple-touch-icon.png     # iOS home-screen icon
    └── manifest.webmanifest         # PWA manifest
```

Live Demo

1. **Login** (demo auth: email + student ID)
2. **Calendar day view**: swipe/tap days, show free/busy blocks
3. **Filters**: capacity ≥ 4, equipment, time window 16:00–18:00, next 7 days
4. Select “Room Neptune” free slot → **booking drawer opens**
5. Add attendees → confirm → **success screen**
6. Show **calendar updates immediately**
7. Open **My bookings** → cancel → show it disappears
8. Show **email confirmation/cancellation** (screenshot is fine if live email is risky)
9. Show “conflict warning” by attempting a booking that overlaps.

Responsiveness + PWA

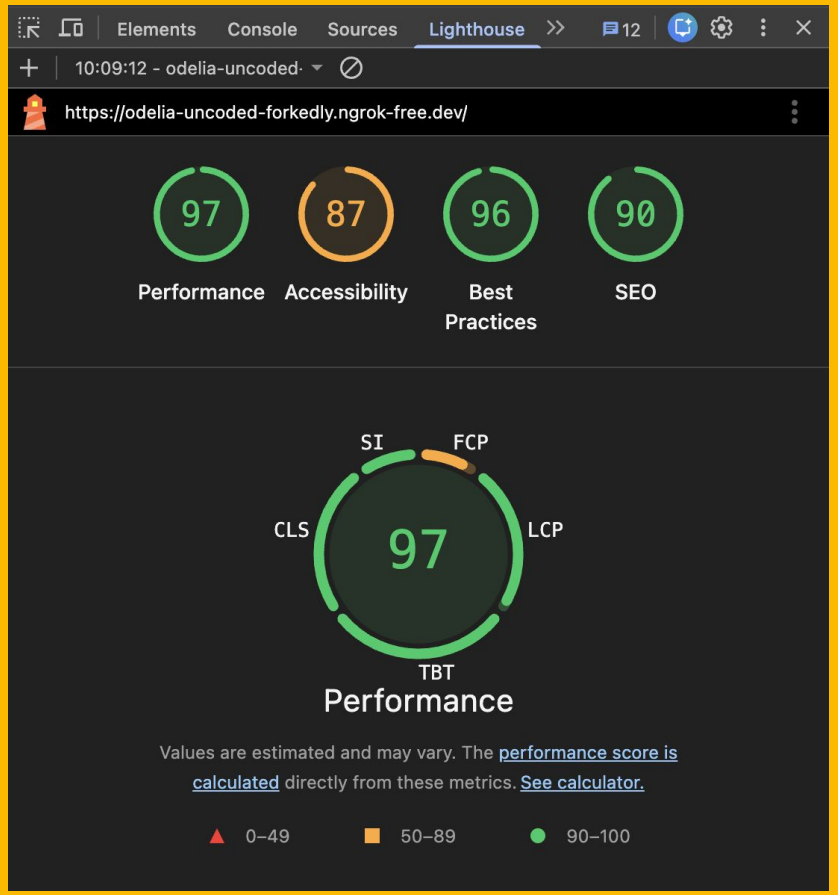
Responsiveness (mobile-first)

- Built with **Bootstrap grid + utility classes** → works from phone to desktop
- **Touch-friendly UI**: big buttons, clear labels, minimal typing (tap slot → book)
- **Readable day view** on small screens: horizontal swipe/scroll across room columns
- **Filters accordion** keeps the UI clean on mobile (expand only when needed)
- Tested with **phone-sized viewport + rotation** (portrait + landscape)

PWA components implemented

- **Web App Manifest** (manifest.webmanifest)
  - name/short_name, theme colour, icons (192/512), display: standalone
- **Service Worker** (/sw.js)
  - caches basic “app shell” (HTML/CSS/JS/manifest)
  - offline fallback for the UI shell
- **Install works**
  - iOS: Share → **Add to Home Screen** (uses apple-touch-icon)
  - Android/Chrome: **Install app** prompt / menu option

Lighthouse

Improvements I made:

- **PWA installability**: added manifest + service worker registration
- **Offline shell**: cached core assets so the app still opens without network
- Kept **frontend lightweight**: minimal JS + Bootstrap only